

Dynamic Scene Graph Updater Using Answer Set Programming

Toya Takahashi

May 13, 2026

1 Introduction

The Dynamic Scene Graph (DSG) stores the robot’s semantic understanding of its environment as a graph in Neo4j. As the robot acts in the world, the graph must be kept internally consistent: objects that are picked up should track the robot’s position, objects stacked on top of other objects should inherit their pose, and relationships that are no longer valid should be retracted. This project’s purpose is to build a rule-based maintenance layer on top of the DSG using Answer Set Programming (ASP) via Clingo, enabling humans and LLMs to declare consistency rules in a uniform language that is automatically enforced on every update cycle.

Rather than writing rules as Python functions that issue hard-coded Cypher queries, we use the Clingo ASP system for a couple of reasons:

1. **Scalability:** Without Clingo, adding a new rule requires writing Python and Cypher by hand. There is no shared language in which a human or an LLM can describe what a rule should do.
2. **Consistency:** A rule that derives new edges in the scene graph database may conflict with another rule that deletes them. Running independent Cypher queries does not detect such conflicts; the graph can end up in an inconsistent state.

With ASP, rules are written in a uniform declarative language, and the solver guarantees that the derived state is consistent before mutations are committed.

2 Usage

Launch the system using the `heracles_prior_dsg` configuration, which starts the default ASP rules example. Once the Spot is manually moved to a location inside the DSG map, prompts can be sent to the ChatDSG LLM. A list of videos of working demonstrations can be found [here](#).

2.1 Pick, Place, and Move

A working sequence of prompts is:

1. `can you pick up 027`
2. `can you place 027 at P26231`
3. (if needed) `the place exists as a mesh node`
4. (if needed) `027 is already picked up`
5. `can you move to 088`

This sequence shows the robot pick up an object, place it, and move away, demonstrating that the object was placed. A video example can be found [here](#).

2.2 Stacking and Unstacking

Objects can also be stacked and unstacked within the same ASP example. Example prompts:

- 0285 is not stacked on top of 027
- 0285 is stacked on top of 027

This works because of the updated LLM prompt in `cypher_interface_holding_description.yaml`.

3 Architecture

Each update cycle follows three steps.

1. **Snapshot:** The current graph state is queried from Neo4j and converted into a set of ASP ground facts. For example,

```
robot("hamilton").
object("box1").
object("cup1").
class("box1", "box").
class("cup1", "cup").
holds("hamilton", "box1").
with("box1", "cup1").
```

2. **Solve:** The snapshot facts are combined with an ASP rule program and handed to Clingo. The solver computes a set of derived atoms that satisfies all rules and constraints. If any constraint is violated, the solver returns UNSAT and no mutations are applied.
3. **Translate:** Each action atom in the stable model is looked up in a dictionary of `ActionTemplate` objects. Each template is a Cypher query string with named placeholders that are substituted with the atom's arguments and executed against Neo4j.

4 Example Rules

Example sets of rules can be found in `dsg_updater/examples/asp_rules_example.py`.

4.1 Object Holding Rule

When a robot holds an object, the object's position in the graph should equal the robot's position.

```
update_position(0, R) :- holds(R, 0), robot(R), object(0).
```

The action atom `update_position(0, R)` instructs the translator to set the position of object 0 to the current position of robot R.

4.2 Object Stacking Rule

When an object Y is stacked on top of object X (i.e. there is a `WITH` edge $X \rightarrow Y$), Y's position must be recomputed relative to X whenever X moves.

```
update_stack_position(Y, X) :- with(X, Y), object(X), object(Y).
```

The action atom `update_stack_position(Y, X)` instructs the translator to reposition Y on top of X's bounding box.

4.3 Belief Space Example

The belief space example, found in `dsg_updater/examples/asp_belief_example.py`, tracks object state across pick-up and placement events using three rules.

4.3.1 Pickup Tracking

On the first cycle where a **HOLDS** edge appears for an object, two atoms are derived and persisted into Python memory for future cycles.

```
has_pickup(O)           :- picked_up(_, O).
add_picked_up(O, R)     :- base_holds(R, O), not has_pickup(O).
remember_origin(O, X, Y, Z) :- base_holds(_, O), center(O, X, Y, Z), not has_pickup(O).
```

`add_picked_up(O, R)` records that robot `R` has held object `O` at some point. `remember_origin(O, X, Y, Z)` captures the object's position at the moment of pick-up. Both are accumulated into a Python `set` and injected as ground facts on every subsequent cycle. The guard `not has_pickup(O)` ensures these fire only once per object.

4.3.2 Most Probable Position

While a **HOLDS** edge exists, the most probable state is that the robot successfully has the object. The position is updated deterministically each cycle. This is equivalent to the Object Holding Rule described above.

```
update_position(O, R) :- base_holds(R, O), robot(R), object(O).
```

4.3.3 Belief Space

The belief rule enumerates all possible object states using choice rules:

```
held(O)           :- held_by(O, _).
ever_picked_up(O) :- picked_up(_, O).
currently_held(O) :- base_holds(_, O).

% while HOLDS edge exists: robot may or may not actually have the object
{held_by(O, R)}   :- base_holds(R, O).
at_origin(O, X, Y, Z) :- currently_held(O), not held(O), origin(O, X, Y, Z).

% after HOLDS removed: object may have been placed or dropped
{placed_at(O)}    :- ever_picked_up(O), not currently_held(O).
at_origin(O, X, Y, Z) :- ever_picked_up(O), not currently_held(O),
                        not placed_at(O), origin(O, X, Y, Z).
```

While a **HOLDS** edge is present, the choice rule on `held_by` produces two worlds per held object: one in which the robot has it, and one in which it does not. After the **HOLDS** edge is removed, the choice rule on `placed_at` produces two worlds: the object was placed at the new location, or it was dropped and is back at its origin. With n objects in an uncertain state, 2^n worlds are produced in total.

5 Types of Rules to Specify

5.1 Labeling and Classification

- **Region-based labeling:** If an object is located inside a semantic region (e.g. a room or zone), it can be assigned a label derived from that region.
- **Bulk reclassification:** All objects of class `X` are relabeled to class `Y`. This can be useful for correcting systematic misclassifications from the perception system.
- **Proximity-based class correction:** Two overlapping objects of semantically confused classes are reclassified to a canonical class, or merged if they share the same class.

5.2 Cardinality and Mutual Exclusion

The following can be expressed as integrity constraints:

- **At most one object held:** A robot holds at most one object
- **No objects of label `X` exist:** Assert that a particular class should never appear in the graph

5.3 Integrity Constraints

These rules assert invariants that must hold before any mutations are committed. If a constraint fires, the solver returns UNSAT and the update cycle is skipped entirely.

```
% A HOLDS edge must point to a known robot.
:- holds(R, _), not robot(R).

% WITH edges must not form self-loops.
:- with(X, X).

% An object cannot be held by two robots simultaneously.
:- holds(R1, O), holds(R2, O), R1 != R2.

% At most one object is held by one robot.
:- holds(R, O1), holds(R, O2), O1 != O2.

% A particular class should never appear.
:- class(_, "X").
```

5.4 Discrete Belief With Multiple Possible Answer Sets

Rather than estimating a continuous probability distribution over object states, we maintain a finite set of candidate world states. Each stable model corresponds to one hypothesis. An example implementation is described in Section 4 and in `dsg_updater/examples/asp_belief_example.py`.

6 Potential Problems

Using ASP rules is a good choice for structural and relational reasoning over a static snapshot at a certain point in time, but there are some drawbacks in the current system, along with rules which are difficult to encode.

6.1 Re-derivation of Rules Each Cycle

ASP assumes that anything not derivable is false. Rules must be re-evaluated from scratch each cycle, which is a performance bottleneck and will discard any fact derived in the previous cycle that is not re-derivable from the current snapshot.

6.2 Multiple Acceptable Solutions

A rule program can produce more than one answer set, especially if a choice rule is used. In the current implementation the solver returns the first model found, which is arbitrary. Integrity constraints can eliminate unwanted models but cannot guarantee a unique solution.

6.3 Temporal Reasoning

Rules that require temporal reasoning, such as “if an object has not moved in 5 seconds, mark it as placed,” are not expressible in the current architecture. ASP operates on a single snapshot with no notion of time across cycles.

6.4 Probabilistic Rules

ASP is strictly boolean. Uncertainty, confidence scores, and weighted preferences cannot be expressed directly.